Oturum 1 Notları

Pointer, referans, `const` ve `const&` parametre

Dosya	Konu
`01-temeller/01_swap.cpp`	Değerle / referansla / pointer ile geçirme: swap
`01-temeller/02_const_pointer.cpp`	const int* ile int* const farkı
`01-temeller/03_const_ref.cpp`	const& parametre: kopyasız ve güvenli okuma

Bu notta her konu aynı sırayla işlenir: **soru, kod, doğru cevap, sık yapılan hata, mülakat cümlesi.** Hedef: Enetki Savunma staj teknik mülakatı (26 Ekim 2026).

01 · Pointer ve referans: swap

Soru 1: p ile r arasındaki fark

```
int x = 10;
int* p = &x;   // p: x'in ADRESINI tutan ayri bir degisken
int& r = x;    // r: x'in baska ADI (takma ad), ayri bir sey degil
```

- **&x** ifadesinde **&** "adresini al" demektir. **int&** tür tarafında "referans" demektir. Aynı işaret, iki ayrı anlam.
- ***p** "p'nin gösterdiği yerdeki değer" demektir.
- Pointer null olabilir ve sonradan başka yere yöneltilebilir. Referans tanımlanırken bağlanmak zorundadır, null olamaz, sonradan başka nesneye bağlanamaz.

Mülakat cümlesi: "Pointer, adres tutan ayrı bir değişkendir; null olabilir ve sonradan başka yere yöneltilebilir. Referans ise bir nesnenin takma adıdır; tanımlanırken bağlanmak zorundadır, null olamaz ve sonradan başka nesneye bağlanamaz."

Soru 2: çıktı ne olur?

```
int x = 10;
int* p = &x;
int& r = x;

*p = 20;
r = 30;
std::cout << x << "\n";
```

Doğru cevap: 30. r, x'in kendisidir; `r = 30` yazmak ile `x = 30` yazmak birebir aynıdır.

Sık yapılan hata: "r kendi 30'unu tutuyor, x 20'de kalıyor" demek. Bu, referansı kopya sanmaktır.

Kontrol: `int r = x;` (& yok) olsaydı?

```
int x = 10;
int r = x;     // KOPYA: r = 10, simdilik
x = 20;        // x = 20 olur, r etkilenmez (10)
r = 30;        // sadece kopya degisir, x hala 20
// cikti (x): 20
```

Kural: kopya, o satırdaki değeri **dondurur**. Sonradan x değişse de kopya haberdar olmaz. Referansta tek nesne iki isimle durur; kopyada iki ayrı nesne olur.

Soru 3: swap neden çalışmadı?

```
void swapValues(int a, int b) {   // a ve b, cagiran taraftaki x ve y'nin KOPYASI
    int temp = a;
    a = b;
    b = temp;
}
```

Sebep: değerle geçirme (pass by value). Fonksiyon kopyaları takas eder, orijinale hiç dokunmaz. Global değişken kullanmak çözüm değildir (kodu bozuk ve hataya açık yapar). Çözüm: fonksiyona kopya yerine **orijinale ulaşma yolu** vermek.

Çalışan hâli (dosya: 01_swap.cpp)

```
// Konu: Değer değiştirme (swap): referans ve pointer ile
// Ne gösteriyor: Aynı işi yapan iki fonksiyon; biri referans, diğeri pointer alıyor.
// Neden önemli: "Değerle geçirme" kopya üretir; orijinali değiştirmek için referans veya pointer gerekir.
// Derleme: g++ -std=c++17 -Wall -Wextra -o 01_swap 01_swap.cpp

#include <iostream>

// Referansla: a ve b, çağıran taraftaki değişkenlerin takma adıdır.
void swapRef(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

// Pointer ile: a ve b adres tutar; değere ulaşmak için * kullanılır.
void swapPtr(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 3;
    int y = 7;

    std::cout << "Basta:        x=" << x << " y=" << y << "\n";

    swapRef(x,y);
    std::cout << "swapRef sonra: x=" << x << " y=" << y << "\n";

    swapPtr(&x,&y);
    std::cout << "swapPtr sonra: x=" << x << " y=" << y << "\n";

    return 0;
}
```

Çağrı yazımı: referans ve pointer

	swapRef (referans)	swapPtr (pointer)
Tanım	`void swapRef(int& a, int& b)`	`void swapPtr(int* a, int* b)`
Çağrı	`swapRef(x, y);`	`swapPtr(&x, &y);`
İçeride değere erişim	a, b (doğrudan)	*a, *b
Parantez içine ne yazılır	değişkenin kendisi	değişkenin adresi

Sık yapılan hata: tanımdaki parametre yazımını çağrıya kopyalamak (`swapRef(int &x, int &y);`). Tür yazmak yalnızca yeni değişken **tanımlarken** gerekir; çağırırken elindeki değişkenlerin adını verirsin.

Pointer sürümünde * ve & karışıklığı

Yazım	Anlamı	Örnek
&x	x'in adresini ver	`int* p = &x;`
*p	p'nin gösterdiği yerdeki değeri ver	`*p = 20;`

Fonksiyonun içinde a ve b zaten adres olduğu için **& kullanılmaz, sadece * kullanılır**. * yalnızca pointer olan şeylere uygulanır; `temp` düz int olduğu için `*temp` yazılamaz.

Bellekte swapPtr(&x, &y) (x=3, y=7)

Satır	Ne olur?	x	y
(başlangıç) a=1000, b=1004		3	7
`int temp = *a;`	1000'deki değer (3) temp'e kopyalanır	3	7
`*a = *b;`	1004'teki değer (7), 1000'e yazılır	**7**	7
`*b = temp;`	temp (3), 1004'e yazılır	7	**3**

temp'in nedeni: 2. satırdan sonra x'in eski değeri (3) ezilir. temp olmasaydı 3 kaybolurdu. İki takas arka arkaya çalışınca değerler başlangıca döner (3 7 → 7 3 → 3 7); bu hata değil, beklenen sonuçtur.

Pointer mı referans mı?

Mülakat cümlesi: "İki nesnenin de kesin var olduğu durumda referans tercih ederim; null kontrolüne gerek kalmaz ve çağrı yazımı temiz olur. Nesne opsiyonel olabiliyorsa (olabilir de olmayabilir de) pointer kullanırım." Ek bilgi: `swapPtr(&x, &y)` çağrısında &, "bu değişebilir" diye çağıran tarafta bir işaret bırakır; referansta çağrıdan bunu göremezsin.

Derleme ve çalıştırma notları

- g++ hata ve uyarı vermiyorsa **hiçbir şey yazmaz**; bu iyi haberdir. Derlemek programı çalıştırmak değildir.
- Çalıştırma (PowerShell): `.\01_swap` (başa `.\` gerekir).
- "No such file or directory": yanlış klasördesin. Dosyanın olduğu klasöre `cd` ile geç, terminaldeki yola bak.
- Editördeki kırmızı çizgi (IntelliSense) ile derleyici hatası ayrı şeylerdir. Asıl kaynak **g++ çıktısı**dır.
- `.exe` dosyaları derleme çıktısıdır, repoya girmez (.gitignore) ve her derlemede yeniden oluşur.

02 · const ve pointer

Pointer'da iki ayrı şey sabit olabilir: **adres** (pointer'ın kendisi) ve **adresteki değer**. Tanımı **sağdan sola** oku; `const` hemen solundaki şeyi sabitler.

```
const int* p1 = &a;   // p1: "const int"e isaret eden pointer -> DEGER sabit, adres serbest
int* const p2 = &a;   // p2: int'e isaret eden "const pointer" -> ADRES sabit, deger serbest
```

	const int* p1	int* const p2
Değeri değiştir (`*p = ...`)	hata	serbest
Adresi değiştir (`p = ...`)	serbest	hata

Dört satırın cevabı (a=10, b=20)

#	Satır	Sonuç	Neden
1	`*p1 = 99;`	hata	p1 const int*: değere yazmak yasak
2	`p1 = &b;`	derlenir	Sadece adres değişir (p1 artık b'yi gösterir); b'ye dokunulmaz, b hâlâ 20
3	`*p2 = 99;`	derlenir	int* const: değer sabit değil
4	`p2 = &b;`	hata	Pointer'ın adresi sabit

Sık yapılan hata: `*p1 = ...` (yıldız var: **değere** dokunur) ile `p1 = ...` (yıldız yok: **adrese** dokunur) satırlarını karıştırmak.

Çalışan dosya (02_const_pointer.cpp; hata veren satırlar yorumda)

```
// Konu: const ve pointer: const int* ile int* const farkı
// Ne gösteriyor: Pointer'da iki ayrı şey sabit olabilir: gösterilen değer veya pointer'ın kendisi.
// Neden önemli: Mülakatta çok sorulur; ayrıca const& parametreleri gereksiz kopyayı önler.
// Derleme: g++ -std=c++17 -Wall -Wextra -o 02_const_pointer 02_const_pointer.cpp

#include <iostream>

int main() {
    int a = 10;
    int b = 20;

    // Durum 1: const int* (değer sabit, adres değişebilir)
    const int* p1 = &a;
    // *p1 = 99;
    p1 = &b;

    // Durum 2: int* const (adres sabit, değer değişebilir)
    int* const p2 = &a;
    *p2 = 99;
    // p2 = &b;



    // Her denemeden önce tahmin et: derlenir mi, derlenmez mi?
    std::cout << a << " " << b << "\n";
    return 0;
}
```

Çıktı: `99 20` (*p2 = 99 ile a değişti; p1 = &b sadece adresi değiştirdi, b 20 kaldı).

a = 5 ile *p1 = 5 farkı

```
int a = 10;
const int* p1 = &a;

a = 5;       // (A) DERLENIR: a duz bir int, sabit degil
*p1 = 5;     // (B) HATA: p1 uzerinden degere yazmak yasak
```

Önemli: sabit olan **a değil, p1 üzerinden erişim yoludur**. const, değişkene değil o pointer'a bir kısıtlama koyar.

Değişkenin kendisi const ise

```
const int a = 5;
a = 20;               // HATA: read-only variable
int* p = &a;          // HATA: const'i kaldirmaya calisiyorsun
const int* p = &a;    // TAMAM: "bu deger sabit" sozunu pointer da veriyor
```

Tanım	Sabit olan
`const int a = 5;`	değişkenin kendisi (hiçbir yoldan değişmez)
`const int* p = &a;`	p üzerinden erişilen değer
`int* const p = &a;`	p'nin tuttuğu adres

Terim notu: pointer için "gösterir / işaret eder" denir; "referans verir" ifadesi & ile tanımlanan takma ad için ayrılmıştır.

Mülakat cümlesi: "`const int*` ile pointer üzerinden değeri değiştiremem ama pointer'ı başka yere yöneltebilirim. `int* const` ile pointer sabittir, hep aynı yeri gösterir ama gösterdiği değeri değiştirebilirim."

Editörün mesajı "expression must be a modifiable lvalue": *lvalue* bellekte yeri olan ve ='in soluna yazılabilen şeydir. `*p1` bir lvalue'dur ama const olduğu için *modifiable* değildir. Gerçek derleyici mesajı farklıdır (ör. `assignment of read-only location`).

03 · const& parametre

Bir fonksiyona nesneyi üç şekilde verebilirsin. Dosyadaki Tracker sınıfı, kopyalandığında ekrana "KOPYA!" yazar; böylece kopya gözle görülür.

Sürüm	Parametre	Kopya olur mu?	Orijinali bozabilir mi?
(1)	`Tracker t`	evet	hayır (kopyayı değiştirir, orijinal etkilenmez)
(2)	`Tracker& t`	hayır	evet
(3)	`const Tracker& t`	hayır	hayır

(1) güvenli ama pahalı, (2) ucuz ama riskli, (3) hem ucuz hem güvenli: **sadece okuma için en iyi seçim**.

Dosya: 03_const_ref.cpp

```
// Konu: const& parametre: kopyasız ve güvenli okuma
// Ne gösteriyor: Aynı nesneyi üç farklı biçimde (değerle, referansla, const referansla) fonksiyona
    geçiriyor.
// Neden önemli: Büyük nesneleri gereksiz kopyalamamak ve fonksiyonun orijinali bozmasını önlemek için her
    yerde kullanılır.
// Derleme: g++ -std=c++17 -Wall -Wextra -o 03_const_ref 03_const_ref.cpp

#include <iostream>
#include <string>

// Kopyalandığında ekrana "KOPYA!" yazan küçük bir sınıf.
// (Kopya constructor'ı henüz görmedik; şimdilik sadece "kopya olunca bu satır çalışır" diye bil.)
struct Tracker {
    std::string name;

    Tracker(const std::string& n) : name(n) {}
    Tracker(const Tracker& other) : name(other.name) {
        std::cout << "  KOPYA! (" << name << ")\n";
    }
};

// (1) Değerle: her çağrıda kopya oluşur.
void showByValue(Tracker t) {
    std::cout << "  okundu: " << t.name << "\n";
}

// (2) Referansla: kopya yok ama fonksiyon orijinali değiştirebilir.
void showByRef(Tracker& t) {
    std::cout << "  okundu: " << t.name << "\n";
}

// (3) const referansla: kopya yok ve değiştirmek de yasak.
void showByConstRef(const Tracker& t) {
    std::cout << "  okundu: " << t.name << "\n";
}

int main() {
    Tracker original("sensor-1");

    std::cout << "(1) showByValue:\n";
    showByValue(original);

    std::cout << "(2) showByRef:\n";
    showByRef(original);

    std::cout << "(3) showByConstRef:\n";
    showByConstRef(original); // Kopya yazmaz ("KOPYA!" çıkmaz).

    // Deney A: showByConstRef'in içine  t.name = "degisti";  yaz. Derleyici ne diyor?
    // Deney B: showByRef(Tracker("gecici"));  yaz. Derleyici ne diyor? Aynısını showByConstRef ile dene.

    return 0;
}
```

Çıktı

```
(1) showByValue:
  KOPYA! (sensor-1)
  okundu: sensor-1
(2) showByRef:
  okundu: sensor-1
(3) showByConstRef:
  okundu: sensor-1
```

"KOPYA!" yalnızca (1)'de yazıldı: değerle geçirme kopya üretir, iki referans sürümü üretmez.

Deney A: const& içinde değiştirmek

```
void showByConstRef(const Tracker& t) {
    t.name = "degisti";   // <-- eklenen satir
    ...
}
```

```
error: no match for 'operator=' (operand types are 'const std::string' and 'const char [8]')
note: passing 'const std::string*' as 'this' argument discards qualifiers
```

Yorum: Parametre const olduğu için `t.name` da const std::string olur; atama, const sözünü çiğnerdi. "discards qualifiers" = const'ı yok saymak. Aynı hata, `const int* p1` ile `*p1 = 99;` yazınca da çıkar.

Uzun hata nasıl okunur: (1) ilk `error:` satırını oku, (2) "discards qualifiers" gibi anahtar cümleyi ara, (3) alttaki `candidate 1..6`, `basic_string.h`, `enable_if` satırları gürültüdür, okuma. Terminaldeki bozuk Türkçe karakterler (`de─şi┼ştirilemez`) yorumdaki harflerin kodlamasıdır, derlemeyi etkilemez.

Deney B: geçici nesne

`Tracker("gecici")` adı olmayan, o satırda yaratılıp biten **geçici bir nesnedir**.

Çağrı	Sonuç
`showByRef(Tracker("gecici"));`	HATA: cannot bind non-const lvalue reference of type 'Tracker&' to an rvalue of type 'Tracker'
`showByConstRef(Tracker("gecici"));`	Derlenir, çalışır, "KOPYA!" yazmaz: okundu: gecici

Neden: `Tracker&` "bu nesneyi değiştirebilirim" der; satır biter bitmez yok olacak geçici bir nesneyi değiştirmenin anlamı yoktur, derleyici izin vermez. `const Tracker&` "sadece okurum" dediği için güvenlidir ve kabul edilir. Bu yüzden fonksiyon parametrelerinde sadece okuma için `const T&` yazılır; her türlü argümanı (geçici dahil) kabul eder.

string için karar tablosu

İhtiyaç	Parametre
Sadece okumak / aramak	`const std::string& s`
Çağıranın orijinalini değiştirmek	`std::string& s`
Fonksiyon içinde kendi kopyası üzerinde çalışmak	`std::string s`

Dikkat: `const std::string` (& olmadan) yine kopya üretir; önemli olan **const std::string&**.

Mülakat cümlesi: "Sadece okuyacaksam `const std::string&` alırım: kopya maliyeti olmaz ve yanlışlıkla değiştirilemez. Orijinali değiştirmem gerekiyorsa `std::string&`, kendi kopyamla çalışmam gerekiyorsa değerle alırım."

Özet: tekrar edilecek noktalar

- **& ve * iki işaret, iki iş:** &x adres verir; *p adresteki değeri verir. Yıldız varsa değere, yoksa adrese dokunursun.

- **Referans kopya değildir:** `int& r = x;` tek nesne, iki isim. `int r = x;` ise kopyadır ve o satırdaki değeri dondurur.
- **Çağırırken tür yazılmaz;** tanımda yazılır.
- **const solundaki şeyi sabitler;** sağdan sola oku.
- **Sadece okuma:** `const T&`.
- Terminal: g++ sessizse derleme başarılıdır; çalıştırmak ayrı adımdır (`.\ad`); doğru klasörde ol.

Hızlı ateş (30 saniyede cevapla)

- Pointer ile referans farkı nedir?
- `const int*` ile `int* const` farkı nedir?
- Neden `const std::string&` yazıyoruz?
- Değerle geçirmede ne olur, swap neden çalışmaz?
- Neden geçici nesne `T&`'ye bağlanamaz ama `const T&`'ye bağlanabilir?